

Introdução ao Assembly x86-64

Andrei Formiga

November 19, 2025

Sumário

1. Introdução	3
2. Linguagem Assembly	4
3. Modelo de Execução	5
4. Registradores	6
5. Instruções	7
5.1. Cópia/transferência (MOV)	7
5.2. Soma (ADD)	7
5.3. Subtração (SUB)	8
5.4. Multiplicação (IMUL)	8
5.5. Divisão (IDIV)	8
6. Seções	10
7. Hello, World	11

1. Introdução

Os computadores que usamos são muito versáteis e utilizados em um grande número de aplicações, em vários domínios. A chave para essa versatilidade é que os computadores são *programáveis*, ou seja, ao invés de seguir apenas um conjunto pré-determinado de funções (determinada pelo projetista do hardware), os computadores podem ser programados para realizarem diferentes tarefas, todas tendo como base um conjunto relativamente simples de operações realizadas pelos processadores.

O conjunto de operações simples que podem ser executadas diretamente pelo processador é chamada de *linguagem de máquina* do processador. Um programa em linguagem de máquina é representado como uma sequência de bytes na memória, com cada operação sendo associada a um código. Por exemplo, a instrução ADD pode (dependendo da situação) ser codificada com o código 80 em hexadecimal (128 em decimal). Dizemos que 80 é o código da operação ou *opcode* da instrução ADD. Assim, quando o processador está executando código e encontra um byte de valor 80, ele efetua uma operação de adição. O que deve ser adicionado (os operandos) são codificados após o *opcode*.

Para escrever um programa em linguagem de máquina contendo uma operação ADD, o programador deve consultar alguma referência para determinar o *opcode* da operação e também como codificar os seus operandos.

Obviamente, esta é uma forma muito primitiva de programar, embora tenha sido usada em alguns momentos. Rapidamente os programadores pensaram em criar uma forma mais amigável de criar programas, automatizando parte do processo. Dessa ideia surgiram as linguagens de montagem ou *assembly*.

2. Linguagem Assembly

Uma linguagem *assembly* (ou linguagem de montagem) define uma representação textual para um programa em código de máquina. Ao invés de escrever os bytes 0x80 0x10 0x00 para uma operação de soma, o programador pode escrever:

```
ADD $16, %AL
```

Depois de escrever seu programa em um arquivo de texto, o programador vai usar um *assembler* (ou montador) para montar o programa em linguagem de máquina, que pode ser executado.

A sintaxe da linguagem assembly utilizada vai variar de acordo com o conjunto de instruções (*instruction set*) do processador e também de acordo com o montador específico utilizado. Por exemplo, a instrução ADD vista acima está de acordo com a sintaxe padrão do GNU Assembler (gas) para a arquitetura x86-64. Se fôssemos usar o NASM ou MASM a mesma instrução teria uma sintaxe ligeiramente diferente:

```
ADD AL, 16
```

3. Modelo de Execução

Um processador executa diretamente um conjunto de instruções simples, mas nada funciona sem a existência de alguma memória para guardar dados (de entrada, de saída, ou intermediários) e mesmo o código do programa que deve ser executado. (O fato do processador poder executar programas arbitrários que estejam na memória é o que o torna programável).

De forma simplificada, o processador tem acesso a dois tipos de memória: os *registradores*, que são uma memória interna ao processador, e a memória externa. Os registradores possuem espaço limitado mas são muito mais rápidos de acessar que a memória externa (por exemplo, somar dois números em registradores pode ser centenas de vezes mais rápido que somar dois números na memória). A memória externa leva mais tempo para acessar mas tem muito mais capacidade. Outro aspecto é que, como os registradores estão conectados aos componentes da CPU, é muitas vezes necessário usar pelo menos um registrador na maioria das instruções. Por exemplo, não é possível, na maioria das arquiteturas, somar dois números na memória diretamente; pelo menos um deles deve ser copiado para um registrador.

4. Registradores

Os registradores são divididos em registradores de uso geral e registradores especiais. Na arquitetura x86-64 existem 16 registradores de propósito geral, separados em alguns grupos:

- RAX, RBX, RCX e RDX
- RSI e RDI
- RBP e RSP
- Registradores numerados de R8 até R15

A nomenclatura dos registradores é inconsistente por uma questão histórica. Também por questões históricas, algumas instruções podem restringir quais registradores podem ser utilizados, ou mesmo utilizar alguns registradores fixos de forma implícita (veremos exemplos disso nas instruções de multiplicação e divisão com sinal).

Entre os registradores especiais, apenas dois são importantes para nossos objetivos agora: RFLAGS agrupa uma série de *flags* binárias que refletem o resultado da última operação aritmética ou lógica efetuada pelo processador. Essas *flags* são usadas para implementar condicionais.

O outro registrador especial é o RIP, o *Instruction Pointer*. O conteúdo deste registrador é o endereço da próxima instrução que deve ser executada pelo processador. Normalmente não acessamos esse registrador diretamente, apenas através das instruções de salto.

5. Instruções

Os quatro tipos mais importantes de instruções executadas pelo processador são:

1. Operações aritméticas e lógicas
2. Cópia/transferência de dados
3. Verificação de condições
4. Saltos (mudanças na sequência de execução)

Para os compiladores no início da disciplina usaremos a instrução MOV para atribuir valores constantes aos registradores, e as instruções aritméticas.

5.1. Cópia/transferência (MOV)

A instrução MOV é usada para copiar dados de um local para o outro, o que é necessário o tempo todo em programas *assembly*. Os dados podem ser copiados de um registrador para a memória e da memória para um registrador. Também usamos MOV para atribuir um valor constante a algum registrador e para copiar o valor de um registrador para outro. Como a Unidade Aritmético-Lógica do processador está ligada ao banco de registradores, toda operação aritmética ou lógica na arquitetura x86-64 precisa ter pelo menos um registrador usado como operando (em arquiteturas modernas como ARM ambos os operandos precisam estar em registradores).

As formas da instrução MOV que usaremos inicialmente são:

```
mov reg, reg  
mov imm, reg
```

Em ambos os casos, o valor do primeiro operando é copiado para o segundo operando. Por exemplo a instrução

```
mov $42, %rax
```

Coloca o valor 42 em RAX, e a instrução

```
mov %rax, %rcx
```

Copia o valor de RAX para RCX.

5.2. Soma (ADD)

Os dois formatos que vamos usar inicialmente são soma de dois registradores e soma de um valor imediato com um registrador.

```
add reg, reg  
add imm, reg
```

Nas duas formas, o valor do primeiro operando é somado ao valor do segundo operando e armazenado no registrador do segundo operando.

Por exemplo, a instrução `add %rbx, %rax` vai somar RBX a RAX e armazenar o resultado em RAX.

O operando imediato está limitado a constantes de até 32 bits.

5.3. Subtração (SUB)

As instruções de subtração funcionam de maneira similar às instruções de adição. Os dois formatos que usaremos são os mesmos do ADD:

```
sub reg, reg
sub imm, reg
```

Nas duas formas, a operação realizada é calcular o segundo operando menos o primeiro, e armazenar o resultado no segundo.

Por exemplo, a instrução `sub %rbx, %rax` vai efetuar a subtração RAX - RBX e armazenar o resultado em RAX.

O operando imediato está limitado a constantes de até 32 bits.

5.4. Multiplicação (IMUL)

Existem duas instruções principais usadas para multiplicação na arquitetura x86-64: MUL e IMUL. Vamos usar a instrução IMUL na sua forma de dois operandos, que é similar às instruções ADD e SUB.

```
imul reg, reg
imul imm, reg
```

Assim como com ADD e SUB, IMUL vai multiplicar os dois operandos e colocar o resultado no segundo.

Por exemplo, a instrução `mul %rbx, %rax` vai efetuar a multiplicação de RAX por RBX e armazenar o resultado em RAX.

A instrução IMUL faz a multiplicação levando em conta o sinal dos operandos, enquanto MUL faz a multiplicação sem sinal. Ambas as instruções também possuem uma forma com apenas um operando. Vamos usar apenas as duas formas de IMUL mostradas acima.

5.5. Divisão (IDIV)

A instrução IDIV é diferente das anteriores pois ela só possui formas de um operando. Vamos usar apenas a forma

```
idiv reg
```

Esta instrução realiza uma divisão inteira de 128 bits, usando RAX como os 64 bits mais baixos e RDX como os 64 bits mais altos. O valor dado por RDX:RAX é dividido pelo operando especificado, colocando o quociente da divisão em RAX e o resto da divisão em RDX.

Por exemplo, a instrução `idiv %rbx` vai dividir RDX:RAX por RBX e colocar o resultado em RAX (quociente) e RDX (resto).

Normalmente vamos realizar apenas divisões de 64 bits, portanto só precisaremos do valor de entrada e saída em RAX. Nesse caso, é importante garantir que o valor de RDX seja adequado para a divisão.

Para números sem sinal, isso significa que RDX deve conter o valor zero, mas se RAX possuir sinal, o valor zero em RDX iria alterar o sinal do resultado.

Felizmente, a instrução CQO faz a *extensão de sinal* correta de RAX para RDX:RAX.

(CQO significa *Convert Quad to Octa* porque, na nomenclatura da arquitetura x86, uma palavra tem 16 bits, uma palavra dupla (*double word*) tem 32 bits, uma palavra quádrupla (*quad word*) tem 64 bits e uma palavra óctupla (*octa word*) tem 128 bits).

Em resumo, é importante sempre executar uma instrução CQO antes de fazer uma divisão com IDIV.

(O GAS aceita uma instrução IDIV com dois operandos se o segundo operando for RAX, mas isso pode não funcionar em outros *assemblers*).

6. Seções

Um arquivo objeto no formato ELF usado pelo Linux é dividido em seções. Três seções são obrigatórias em todo arquivo objeto:

- TEXT é a seção para código
- DATA é a seção para dados estáticos inicializados
- BSS é a seção para dados estáticos não-inicializados

Todo *assembler* aceita um conjunto de diretivas que não geram código, apenas estabelecem alguma informação sobre o arquivo objeto. Um exemplo são as diretivas para especificar seções. No GAS, a diretiva é `.section`.

7. Hello, World

```
.section .data
hello:
    .ascii "Hello, World!\n"

.section .text
.globl _start

_start:
    mov $1, %rax    # sys_write
    mov $1, %rdi    # stdout
    mov $hello, %rsi # endereco do buffer
    mov $14, %rdx   # numero de bytes
    syscall

    mov $60, %rax    # sys_exit
    xor %rdi, %rdi   # codigo de saida
    syscall
```

O código *assembly* nesse exemplo define duas seções: DATA e TEXT (a seção BSS será necessariamente gerada no arquivo objeto, mas não conterá nada). Na seção de dados inicializados, colocamos a string que será impressa. O rótulo `hello` apontará para o começo dessa sequência de bytes (similar a um *array* em C).

Depois é definida a seção TEXT que vai conter o código. O rótulo `_start` (o ponto de entrada do programa) é declarado como um nome global usando a diretiva `.globl`. Em seguida, o rótulo `_start` em si é definido para o início do código do programa.

Este código utiliza a chamada de sistema `write` do kernel Linux para imprimir a mensagem na tela. O arquivo que receberá a escrita é a saída padrão (stdout). Após preparar os valores adequados nos registradores, a chamada de sistema é realizada com a instrução `SYSCALL`.

Após isso, a chamada de sistema `exit` é usada para instruir o sistema operacional a terminar o programa.